

CSA1402 – COMPILER DESIGN

CO1 – Assessment Tool 3

TECHNICAL PRESENTATION

Future Trends in Compiler Design

Guided By:
Dr N Deepa

Monika.R
192324281

INTRODUCTION

What is Compiler Design?

A compiler is software that translates source code into machine code so computers can execute programs.

Why Future Trends?

- Modern software requires:
- Faster execution
- Better optimization
- Cloud development
- AI-assisted programming
- Multi-core processor support

Example: When you build an Android app, the compiler converts Java/Kotlin code into executable code.

WHY COMPILER TECHNOLOGY IS EVOLVING

Challenges

- Large software projects
- AI applications
- Multi-core processors
- Cloud computing
- Faster app deployment

Need

- Future compilers must:
- Compile faster
- Generate optimized code
- Reduce power consumption
- Improve developer productivity

AI-BASED OPTIMIZATION

What is it?

Artificial Intelligence helps the compiler choose the best optimization techniques automatically.

Benefits

- Faster execution
- Lower memory usage
- Reduced power consumption
- Automatic optimization

Real-Time Example

Google's MLGO uses machine learning with the LLVM compiler to improve optimization decisions.

Another example: Android applications become faster because the compiler learns better optimization strategies.

JUST-IN-TIME(JIT) COMPILATION

Definition

JIT compiles code while the program is running instead of compiling everything beforehand.

Advantages

- Faster execution
- Runtime optimization
- Better performance

Real-Time Examples

- Java Virtual Machine (JVM)
- Android Runtime (ART)
- Web browsers like Chrome use JIT for JavaScript.

Example: Opening Gmail in Chrome becomes faster because JavaScript is compiled during execution.

PARALLEL COMPILATION

Definition

Different parts of a program are compiled simultaneously using multiple CPU cores.

Benefits

- Faster compilation
- Efficient use of multi-core processors
- Reduced build time

Real-Time Example

Large companies like Microsoft and Google compile huge software projects using parallel compilation.

Example: Building the Linux Kernel uses multiple processor cores simultaneously.

CLOUD-BASED COMPILERS

Definition

Compilation happens on cloud servers instead of the local computer.

Advantages

- No software installation
- Access from anywhere
- Powerful cloud hardware

Real-Time Examples

- Replit
- GitHub Codespaces
- Google Colab (for Python)
- OnlineGDB

Example: Students can write and run C programs directly in a browser without installing Turbo C or VS Code.

INCREASING AUTOMATION

Definition

Future compilers automatically detect and fix many programming issues.

Features

- Auto code completion
- Error detection
- Code suggestions
- Automatic optimization
- Security checking

Real-Time Examples

- GitHub Copilot
- Visual Studio IntelliSense
- Android Studio

COMPARISON TABLE

Trend	Purpose	Real-Time Example
AI Optimization	Better optimized code	Google MLGO
JIT Compilation	Faster runtime	JVM,Chrome
Parallel Compilation	Faster build	Linux Kernel
Cloud Compiler	Compile online	Replit
Automation	Reduce coding effort	GitHub Copilot

FUTURE SCOPE

Future compiler technologies may include:

- AI-generated optimized code
- Quantum computing compilers
- Compiler support for autonomous vehicles
- IoT device optimization
- Green computing with energy-efficient compilation
- Example: Self-driving cars require compilers that generate highly optimized and reliable machine code.

CONCLUSION

Key Points

- AI is making compilers smarter.
- JIT improves runtime performance.
- Parallel compilation reduces build time.
- Cloud compilers enable coding anywhere.
- Automation increases developer productivity.
- Final Statement: Future compiler design focuses on speed, intelligence, automation, and cloud accessibility, enabling developers to build faster, smarter, and more efficient software.

THANK YOU